Secure Teleoperation of a Robotic Arm using Computer Vision



Presented by:
Neo Vorsatz

Prepared for:
A/Prof Daniel Ramotsoela
Dept. of Electrical and Electronics Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Mechatronics

October 18, 2026

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced.
3. This report is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as their own work or part thereof.

Signature:.....
M. S. Tšoeu

Date:.....

Terms of Reference

The terms of reference page is an agreement between yourself and your supervisor outlining what is expected of you in your final year project. Please make sure that this is discussed and written at the beginning of your thesis project.

Acknowledgments

I send my thanks to my supervisor, A/Prof Ramotsoela, for keeping me ontrack with completing this project's goals, as opposed to drifting off on a tangent.

I send my thanks to Shaun Beautement for bringing to my attention the Hairy Ball Theorem, and its implications on a static kinematic mapping.

I send my thanks to Lusango Mbongo for developing the initial designs of the 3D printed mount, which holds the Braccio and attached modules.

I send my thanks to an anonymous person for recommending that I look into virtual private networks.

Lastly, I send my thanks to my friends and family, who all indirectly supported this project.

Oh yes- let's not forget Budget Rolls, for being my primary source of nourishment during long days on upper campus. I recommend trying their *chicken tikka avo sandwiches* if given the chance. (I'm not sponsored by them, and don't take on any liability in giving this recommendation.)

Abstract

- Open the **Project Report Template.tex** file and carefully follow the comments (starting with %).
- Process the file with **pdflatex**, using other processors may need you to change some features such as graphics types.
- Note the files included in the **Project Report Template.tex** (with the .tex extension excluded). You can open these files separately and modify their contents or create new ones.
- Contact the latex namual for more features in your document such as equations, subfigures, footnotes, subscripts & superscripts, special characters etc.
- I recommend using the **kile** latex IDE, as it is simple to use.

Contents

1 Introduction **1**

1.1 Background to the study 1

1.2 Objectives of this study 1

1.2.1 Problems to be investigated 1

1.2.2 Purpose of the study 1

1.3 Scope and Limitations 1

1.4 Plan of development 2

2 Literature Review **3**

2.1 Introduction 3

2.2 Overview 4

2.3 Kinematic Mapping 5

2.4 Inverse Kinematics 7

2.5 Computer Vision 10

2.6 Wireless Network 10

3 Design **13**

3.1 Computer Vision 13

3.2 Kinematic Mapping 13

3.2.1 Algorithm 14

3.3 Network 15

3.4 Actuation 15

3.5 Testing 15

4 Results **16**

4.1 Simulation Results 16

4.2 Experimental Results 16

5 Discussion **17**

6 Conclusions **18**

7 Recommendations **19**

A Additional Files and Schematics **25**

B Addenda **27**

B.1 Ethics Forms 27

C Inverse Kinematics Pseudocode **28**

Chapter 1

Introduction

1.1 Background to the study

A very brief background to your area of research. Start off with a general introduction to the area and then narrow it down to your focus area. Used to set the scene [**?**].

1.2 Objectives of this study

1.2.1 Problems to be investigated

Description of the main questions to be investigated in this study.

1.2.2 Purpose of the study

Give the significance of investigating these problems. It must be obvious why you are doing this study and why it is relevant.

1.3 Scope and Limitations

Scope indicates to the reader what has and has not been included in the study. Limitations tell the reader what factors influenced the study such as sample size, time etc. It is not a section for excuses as to why your project may or may not have worked.

1.4 Plan of development

Here you tell the reader how your report has been organised and what is included in each chapter.

I recommend that you write this section last. You can then tailor it to your report.

Chapter 2

Literature Review

2.1 Introduction

The application areas for robotics continues to expand, in both industrial and household contexts. Fully autonomous robots still pose risks in many contexts such as surgery, hence why it's often preferred to have a human in the control loop. Teleoperation refers to the control of machines, vehicles or robots over a distance, allowing human discretion and intuition to be incorporated into these contexts.

Operators may not always be able to receive training on how to operate robotic systems, as formal training can be very time consuming for individuals and for companies. Intuitive forms of teleoperation aim to solve this, by reading an operator's natural movements or gestures and mapping these to corresponding commands (such as setpoints) for the robot systems. These natural movements can be observed with computer vision, which is far more accessible than specialised sensory hardware. Regardless of the data capture method, intuitive teleoperation introduces the challenge of mapping human kinematics to corresponding kinematics for robotic limbs, which often aren't anthropomorphic.

Another risk that arises with teleoperation is weak network security. Malicious actors may wish to hijack robotic systems, steal configuration data, or even hold robotics systems under ransom. Teleoperation systems have realtime performance requirements, hence why implementing network security without introducing significant latency is an ongoing discussion.

2.2 Overview

Receiving input from the operator has been done either via computer vision or a sensor suite attached to the operator's arm and/or hand. In a demonstration done by Mongardi *et al.* [1], success was found in teleoperating the Braccio arm using an armband that read muscle flexes to determine the operator's pose. An artificial neural network was used to convert the flexing data into 8 different poses. In the works of Reyes *et al.* [2], one of the modes of operation used to teleoperate the Braccio arm uses computer vision to detect hand gestures. This offers a discrete set of operator poses, each corresponding to a pre-defined pose for the Braccio arm. They designed the poses for the Braccio arm such that the poses could be sequenced to complete a teleoperation task.

Drăgoi *et al.* [3] teleoperate a robotic arm with similar kinematics to the Braccio arm. They use the OpenCV software to extract the video feed from a built-in camera on a laptop, and Google's MediaPipe Hands software [4] to identify the pose of the operator's hand. MediaPipe Hands provides monocular hand tracking and pose estimation; it uses the feed from a single camera to provide the coordinates (3-dimensional space) of several landmarks on a human hand.
The team addresses kinematic mapping by first having a calibration step (to improve repeatability among different operators), and then using descriptors (such as "hand pitch" and "horizontal displacement") to control individual joints of the robotic arm. Alternatively using end-effector mapping with inverse kinematics was briefly discussed, but this was opted out as it would reduce stability and introduce more modelling overhead. The paper suggests end-effector mapping with IK as a natural next step.
They address stability of the teleoperation system by; (1) low-pass filtering the setpoints of each joint, using a moving-average filter over 3 frames, (2) limiting the framerate of setpoints to 1 per 0.03s, thereby reducing network traffic and high-frequency oscillations in the setpoints, and (3) applying a deadband of $\pm 2^\circ$ to the joint angles, to reduce jitter. Drăgoi *et al.* chose to send commands to the robotic arm over a local HTTP network, where the robot hosts the server using the HTTPServer and BaseHTTPRequestHandler software. They do not address the cybersecurity of the network.
With these decisions, the project found some success, having to trade off accuracy for accessibility. The primary causes of error identified were: (1) inconsistent camera placement and hand visibility, (2) monocular depth ambiguity, and (3) the operator's familiarity with teleoperation. The paper recommends multi-view capture to improve independence of camera placements, and eliminate monocular depth ambiguity. Another recommendation of note is to use task-aware smoothing, giving gross movements less smoothing, and finer movements more.

Chen *et al.* [5] developed a software platform, which they call TeleopLab, to bring intuitive teleoperation into educational contexts. This allowed students at home to perform practical activities, such as laboratory experiments.
They chose for smartphones to be used by the operators (students) because of how common they are. The smartphone's relative position to an origin is determined using camera frames and motion sensing. This displacement is sent to the host computer at the lab using the public and forwarded port of the computer. The network and the commands were set up using the TeleMoMa framework [6].
When controlling the robotic arm, they map the cartesian coordinates of operator's pose to the end-effector of the robotic arm. The inverse kinematics was handled using the TRAC-IK solver [7], which was plugged into the ROS. TRAC-IK runs two inverse kinematics methods simultaneously, and returns whichever converges first, typically within 1 millisecond. The first method is an extension of the Newton-Raphson method that mitigates local minima, and the second is a sequential quadratic programming optimisation.
The overall project was successful in its goal, receiving positive feedback from the students. The learning/improvement rate was notably slower than with other teleoperation systems [8][9][10], but this may be attributed to the fact that the novice operators were younger in age.

The security of networks is rarely discussed in conjunction with other teleoperation subsystems. In articles that do discuss security in teleoperation, the security has been the focus of the paper with little attention on other subsystems.

2.3 Kinematic Mapping

Mapping the kinematics of one device (the master limb) to the kinematics of another device (the slave limb) has generally been achieved by identifying positional characteristics of the master limb, and using that to calculate the corresponding positional characteristics of the slave limb, using a linear transformation.

Joint mapping is where the positions of individual joints from the master limb is mapped to the positions of individual joints on the slave limb. This method works well when the master and slave share very similar kinematics [11]. The mapping Drăgoi *et al.* [3] use is very similar to joint mapping, in which they map the master's positional descriptors to each of the slave's joints.

Another method is **end-effector mapping**, sometimes called "cartesian mapping" (and very similarly "fingertip mapping"). In this mapping method, the coordinates of the master's end-effector are mapped to coordinates for the slave [11]. The positions of joints in the slave are then determined via inverse kinematics.
Wang *et al.* [8] evaluate end-effector mapping, joint mapping, and a unique "steering mapping" (which can be described as a variant of joint mapping) in terms of human-centric measures, such as cognitive load. Compared to the other two, they found that end-effector mapping gave significantly less cognitive load and resulted in less jerking (hesitations and sudden corrections) from the operator. End-effector mapping is also seen implemented in TeleopLab [5].
A variant of end-effector mapping is applied in the mixed-reality project of Su *et al.* [12], in which the displacement required to achieve a desired setpoint is used to determine velocities for each joint as opposed to just their positions.

Pose mapping is where the intention of the master's pose is used to map to a slave-pose with similar intention. This is generally implemented with a discrete set of mapped poses, and interpolating between these poses [13]. Machine learning computer vision is a popular method of identifying the pose of the master limb. Pose mapping has unpredictable non-linear behaviour for poses that were interpolated [11][9].

Meeker *et al.* [9][10] propose **subspace mapping**, which uses an intermediate subspace to map between dissimilar kinematics. The intermediate subspace has basis vectors corresponding to the minimum number of parameters required to describe a pose, such as the "openness", "spread" and "curl" of a human hand. In their later work [10], they describe a methodology for generating the transformation matrices (and other mapping constants) through a series of experiments as opposed to by intuition.
This mapping model was compared to the end-effector and joint mapping models, with respect to how quickly novice operators were able to complete tasks, and how few tries were needed. They found that subspace mapping out-performed end-effector and joint mapping in these metrics. Joint mapping followed closely, in terms of performance, and this was attributed to the fact that joint mapping offered individual control of each robotic joint, which is not necessarily possible with the proposed subspace mapping.
Subspace mapping performs well in these tasks where spatial positioning is not the challenge. Meeker *et al.* intentionally used a different cartesian mapping method to position the end-effector (hand), although this method wasn't discussed in any depth. Using a different mapping method was possibly due to the limited expressiveness that comes with using a low-dimensional intermediate subspace.

The Braccio arm has significantly dissimilar kinematics to that of a human forearm, making joint-mapping less preferable although still plausible, with the variation used by Drăgoi *et al.* [3]. The Braccio also has a relatively simple end-effector; a two-finger gripper (or "claw"). This excludes the primary advantage that subspace mapping has over other mapping methods. Between end-effector mapping and pose mapping, end-effector mapping has been applied significantly more and consistently shown to be effective, making it the preferable option.

2.4 Inverse Kinematics

Given that end-effector (EE) mapping is the desired way to transform between the kinematics of a human forearm with that of the Braccio, a suitable inverse kinematics (IK) algorithm is required. Among IK solvers, there are analytical and numerical methods.

Analytical methods commonly use geometry, like the rule of cosines, to solve for joint angles. These methods are typically the fastest and most accurate [14][15][16] but are limited, because they assume exactly one closed-form solution exists, or a small finite number of solutions [17], among which one is implicitly selected.
When there are more degrees of freedom than constraints, there are infinitely many solutions. Assuming segment lengths are fixed, Figure 2.1 (a) shows a desired EE position in 2 dimensional space (2 constraints), with the only IK solution. We ignore the mirrored solution, because it is usually implicitly discarded when formulating the close-form solution. In diagram (a), analytical methods work. Diagrams (b) and (c) show two of the infinite possible solutions for the same desired EE position, but with 3 degrees of freedom.
Another challenge for analytical solutions is scenarios with no solutions. Such scenarios arise when the desired EE position is outside of the reachable workspace, either due to limited reach or motion constraints on the degrees of freedom.

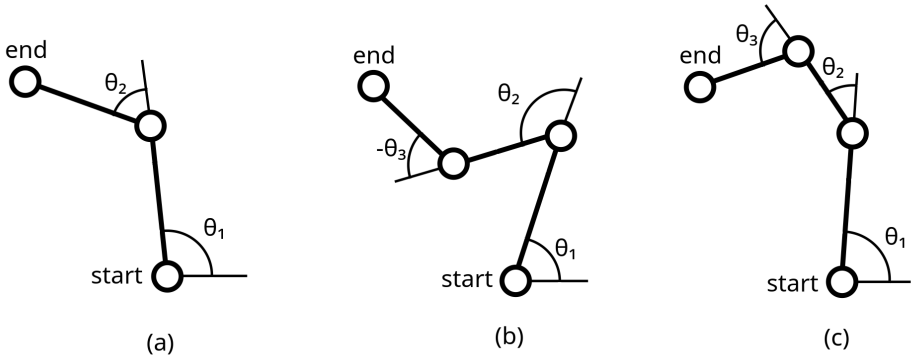


Figure 2.1: Diagram showing IK solutions for different degrees of freedom

Numerical methods typically use an initial guess, then an iterative process to improve that guess until the theoretical EE position is within acceptable error of the desired EE position. These methods handle the cases with infinite or no solutions, but are more computationally expensive.

The most popular ways to perform numerical IK are **Jacobian-based** methodologies [14][15]. By using a system of equations that describe the forward kinematics, the Jacobian for the EE position with respect to the actuations is calculated, giving the relationship shown in Equation 2.1. The Moore-Penrose pseudo-inverse of this Jacobian is used in a Newton-Raphson iterative process, where successive best guesses are generated [14], shown in Equation 2.2.

$$\frac{d\vec{x}}{dt} = J(\vec{q}) \cdot \frac{d\vec{q}}{dt} \tag{2.1}$$

$$\vec{q}_{i+1} = \vec{q}_i + J(\vec{q}_i)^\dagger \cdot (\vec{x}_{target} - \vec{x}(\vec{q}_i)) \tag{2.2}$$

Newton-Raphson struggles with singularities, which occur when no solution exists, or the initial guess leads to a local maximum/minimum that does not have a nearby solution. To counter this, a Damped Least Squares procedure can be implemented, which replaces the pseudo-inverse [14], as shown in Equation 2.3. b is a small positive damping factor, and I is the identity matrix.

To meet requirements other than EE position, these Jacobian methods add "secondary tasks", which do not affect the position of the EE, and only the intermediate joints. The **gradient projection method** (GPM) [18] establishes an objective function $H(\vec{q})$ that we want to minimise, and the gradient of this objective function is projected into the null space (where it doesn't affect the EE). This contribution is added to the Newton-Raphson equation, as shown in Equation 2.4. k is a positive scalar that increases the aggressiveness of the objective function. Secondary objectives can include: avoiding joint-limits, or avoiding obstacles by modelling the physical space as a potential field.

$$J^\dagger_{DLS} = J^T(J \cdot J^T + b^2 \cdot I)^{-1} \tag{2.3}$$

$$\vec{q}_{i+1} = \vec{q}_i + J(\vec{q}_i)^\dagger \cdot (\vec{x}_{target} - \vec{x}(\vec{q}_i)) + k \cdot (J(\vec{q}_i)^\dagger \cdot J(\vec{q}_i) - I) \cdot \nabla H(\vec{q}_i) \tag{2.4}$$

The mixed-reality project [12] makes use of a velocity-based variant of the Jacobian method. Instead of computing incremental changes and iterating through the Newton-Raphson method, they used the result from applying the pseudo-inverse as a velocity for the joints.

Forward And Backward Reaching Inverse Kinematics (FABRIK) is a popular type of heuristic numerical method, in which the positions of each joint is iteratively adjusted in a backward (from EE to base) and then forward (base to EE) manner. This is computationally cheaper and easier to implement than gradient methods [17], but under very high precision requirements FABRIK struggles to converge.
FABRIK is also very versatile, with a lot of research done into variations for solving different problems. It can be used to solve for multiple EEs. It has also been extended to solve in scenarios where the kinematics are not like a tree, but instead have loops.
Sequential Quadratic Programming is a Lagrange-based iterative method [19] that naturally incorporates secondary optimisations. It optimises a non-linear scalar function, given a vector constraint. SQP converges faster than heuristic methods, but is sensitive to the initial guess. Xu *et al.* [16] combine FABRIK with SQP by having FABRIK generate the initial guess, and using SQP to converge to a solution.

Cyclic Coordinate Descent (CCD) is another heuristic numerical method, but is designed for serial kinematic chains and thus is difficult to effectively implemented for more generalised robotic systems. It also returns solutions that aren't typically considered natural [17].

Many other numerical methods exists, generally treating IK as a non-linear optimisation problem, and using algorithms such as Riemannian optimisation or the derivate-free optimisation [16]. These are not discussed because of their lack in popularity. Some research has also been done in using neural networks to perform parts of computation, thereby reducing reliance on knowledge of the robot's kinematic structure [16][20]. With the aim of extending Jacobian-based methods to soft robots, Fang *et al.* [21] implemented a neural network to learn the Jacobian computation, and translate the abstractions from simulation into actuation.

Keeping flexibility for future work and improvement in mind, FABRIK is the preferred IK implementation. It offers a very versatile foundation with fast convergence times. For the level of precision on the Braccio, FABRIK's lack of precision won't pose a concern. The particular implementation of FABRIK needed would have joint limit constraints and possibly secondary objectives.

2.5 Computer Vision

Considering that the desired kinematic mapping method is end-effector mapping, a suitable computer vision pipeline would return the cartesian coordinates of an operator's fingertips and the global coordinate of their hand. This rules out computer vision pipelines that perform gesture recognition, such as in the works of Reyes *et al.* [2] and Simão *et al.* [13]. This also narrows the available computer vision pipelines to those that use joints as the abstraction level, as opposed to other abstractions such as silhouettes, edges, blobs, et cetera [22].

Moeslund and Granum [22] make note of the common assumptions made in computer vision. Assumptions they list which are acceptable for this project are: *(i) The subject (human arm) remains inside the workspace. (ii) There is no camera motion. (iii) Only one subject (human arm) is in the workspace at the time. (iv) There is no occlusion of the subject. (v) The motion pattern (kinematics) of the subject is known. (vi) There are minimal changes to lighting. (vii) The background is static/uniform. (viii) The camera parameters are known.*
With these assumptions in mind, the robustness of the computer vision pipeline is not as significant of a priority as accuracy and speed.

Monocular depth ambiguity is a common challenge in computer vision, especially when setups with depth cameras and/or multiple cameras are expensive or difficult to configure. Drăgoi *et al.* [3] faced monocular depth ambiguity as one of the primary contributions to error in their teleoperation system. They used the apparant hand size (calculated via distance between landmarks) as a proxy for depth. Another common method for depth estimation is to use deep machine learning directly [23], such as with denoising diffusion models [24].

2.6 Wireless Network

TODO: Update this section

The network security, performance and reliability (referred to as the "trilemma" by Gounine *et al.* [25]) are a significant concern in teleoperation systems, especially where teleoperation is implemented in medical applications such as telesurgery (teleoperation for surgery) [26]. Due to the very strict regulations surrounding it, telesurgery has its own dedicated software security frameworks, such as SecureSurgiNET [27]. SecureSurgiNET uses a central telesurgical authority (TSA) to manage and verify certificates in their

specialised public key infrastructure (PKI), similar to the X.509 standard.

Mahmoud *et al.* [28] discuss various cyber threats to teleoperation systems. Unencrypted networks are vulnerable to *eavesdropping*, *message modification*, and *false data injection*. Networks without authentication are vulnerable to man-in-the-middle attacks, such as *replay*, *delay*, message dropping, and *spoofing*. The **Secure and Statistically Reliable UDP** (SSR-UDP) protocol is often used as an implementation for both encryption and authentication, which is an adaptive information coding method. *Jamming* attacks on wireless networks, cyber attacks on the master or slave software, *physical* attacks on the hardware, and *social engineering*, all remain a threat.

The default communication middleware for ROS2 is **Data Distribution Service** (DDS), which implements encryption, authentication, and quality of service (QoS). There have been many recent vulnerabilities found in DDS, such as allowing compromised nodes to retain access, even after revoking its certificate [29]. This makes ROS2's DDS not a viable option for sensitive teleoperation systems.

Boston Dynamic's Spot often serves as a cybersecurity benchmark, having very mature security which surpasses that of almost all commercial quadruped robots [29]. Spot's security implements encrypted communications (TLS 1.2+), authentication (X.509 and ES256), firmware integrity, and per-device unique encryption keys.

Takanashi *et al.* [30] successfully develop an encrypted four-channel bilateral control scheme. They implemented homomorphic encryption, which allowed them to do some computation while the data is still encrypted, to enhance security. BFV encryption introduces a lot more quantization than ElGamal encryption, in exchange for more operations that can be done with the ciphertext. They found that this made ElGamal encryption have a better performance-to-security balance.

Virtual Private Networks (VPNs) are a popular approach to network security and privacy. Among VPN protocols, such as OpenVPN, Tailscale and ZeroTier, **WireGuard** has the best performance [31]. This advantage also extends to IoT devices [32]. On an ESP32, the resource consumption of WireGuard is within operational limits, but WireGuard's encryption algorithms introduce $\approx$ 600ms of one-way latency [33]. This makes it suitable for logging and telemetry contexts, but less so for realtime control.

The slave-side networking of this project is handled by an ESP-WROOM-32 [34], limiting the security capabilities. This makes latency, due to overhead, a larger concern compared to the packet success rate. For this reason, SSR-UDP is the preferred protocol over

TCL (which is built on TCP). This project doesn't stand to gain much security from homomorphic encryption, so instead **advanced encryption standard** (AES) algorithms is used, because the ESP-WROOM-32 has dedicated hardware accelerators for AES.

Chapter 3

Design

3.1 Computer Vision

The computer vision pipeline of choice was inspired by Drăgoi *et al.* [3]. OpenCV was used to extract the live video feed of the laptop's camera, which was then fed into Google's MediaPipe Hand Landmark Detection [35]. MediaPipe Hand Landmark Detection provides monocular hand tracking and pose estimation, and is the successor to MediaPipe Hands, as MediaPipe Hands is now deprecated.

Other computer vision pipelines that were considered include Ultralytics' YOLO [36] and MMPose. The **YOLO models** are extremely customisable and easily trainable, making them excellent for niche computer vision tasks such as forensics [37], and overall hand detection, given a good data set [38]. There are no pretrained models for hand pose estimation, and the time constraints of this project did not allow for training a custom model. **MMPose** has pretrained models for hand pose estimation, but isn't as accurate as MediaPipe for this specific task [39].

3.2 Kinematic Mapping

The inverse kinematics uses the FABRIK algorithm. In particular, it implements an adaptation of FABRIK described by Moreno *et al.* [40], which incorporates movement for more types of joints (not just ball joints, like in the original FABRIK), as well as constraints on range of motion. The pseudocode for the algorithm can be seen in Algorithm 1, while the pseudocode for each type of joint can be seen in Appendix C.

3.2.1 Algorithm

This implementation uses a linked list structure to describe the kinematics and positions of an arbitrary serial manipulator. Like with normal FABRIK [41], the algorithm iterates through this linked list structure, adjusting positions of joints under the constraints of its previous joints. This process is repeated for both a forward-reaching and backward-reaching stage. The adapation of FABRIK restricts where successive joints can be positioned based on the type of joint that came before, and then corrects the position based on the joint's range of motion, such as [-90°, 90°].

Algorithm 1 FABRIK Algorithm

Require: An array J of joints where $J[0]$ is the base, $J[n-2]$ is the end-effector, and $J[n-1]$ is the target, an acceptable squared error e, and a maximum number of forward-backward iterations m
Ensure: The end-effector is within e of the target, or iterations reach maximum

function FABRIK(J, e, m)
    // Loop until error reduced or iterations maxed
    $error \leftarrow |J[n-1].\vec{pos} - J[n-2].\vec{pos}|^2$
    $error \leftarrow error + |J[n-1].\vec{axis}_{link} - J[n-2].\vec{axis}_{link}|^2$
    $error \leftarrow error + |J[n-1].\vec{axis}_{aux} - J[n-2].\vec{axis}_{aux}|^2$
    $i \leftarrow 0$
    while $i < m$ **and** $error > e$ **do**
        // Forward reaching
        for $j \leftarrow (n-2)$ to 1 **do**
            if $J[j+1]$ is a hinge **then**
                ForwardHinge(J, j)
            else if $J[j+1]$ is a pivot **then**
                ForwardPivot(J, j)
            else if $J[j+1]$ is prismatic **or** $J[j+1]$ is immovable **then**
                ForwardPrismatic(J, j)
            end if
        end for

        // Backward reaching
        for $j \leftarrow 1$ to $(n-2)$ **do**
            if $J[j-1]$ is a hinge **then**
                BackwardHinge(J, j)
            else if $J[j-1]$ is a pivot **then**
                BackwardPivot(J, j)
            else if $J[j-1]$ is prismatic **or** $J[j-1]$ is immovable **then**
                BackwardPrismatic(J, j)
            end if
        end for

        // End iteration
        $error \leftarrow |J[n-1].\vec{pos} - J[n-2].\vec{pos}|^2$
        $error \leftarrow error + |J[n-1].\vec{axis}_{link} - J[n-2].\vec{axis}_{link}|^2$
        $error \leftarrow error + |J[n-1].\vec{axis}_{aux} - J[n-2].\vec{axis}_{aux}|^2$
        $i \leftarrow i + 1$
    end while
end function

3.3 Network

3.4 Actuation

3.5 Testing

Chapter 4

Results

These are the results I found from my investigation.

Present your results in a suitable format using tables and graphs where necessary. Remember to refer to them in text and caption them properly.

4.1 Simulation Results

4.2 Experimental Results

Chapter 5

Discussion

Here is what the results mean and how they tie to existing literature...

Discuss the relevance of your results and how they fit into the theoretical work you described in your literature review.

Chapter 6

Conclusions

These are the conclusions from the investivation and how the investigation changes things in this field or contributes to current knowledge...

Draw suitable and intelligent conclusions from your results and subsequent discussion.

Chapter 7

Recommendations

This paper only evaluates the kinematic mapping software with functional metrics, but does not provide direct insight into its intuitiveness. I would like to see the kinematic mapping software evaluated on intuitiveness, likely using the *completion rate* and *completion time* of teleoperation tasks, performed by novice operators. These metrics are used to evaluate intuitiveness in other teleoperation systems [3][9][10].

Bilateral teleoperation is where the operator receives tactile feedback from the slave limb. This can improve intuitiveness, but is not explored in this paper. Future work may introduce tactile feedback into the system, similar to some other teleoperation systems [8][11].

Path tracking is a natural next step when discusses inverse kinematics. The kinematic mapping software can be extended to solve for trajectories, in cases where autonomous movement is required or when setpoints are infrequent.

Bibliography

[1] A. Mongardi, F. Rossi, A. Prestia, D. Demarchi, and P. M. Ros, "Live demonstration: A wearable armband for real-time control of multi-dof robotic actuators," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–1. [Online]. Available: https://doi.org/10.1109/ISCAS46773.2023.10181982

[2] A. Reyes, S. Reinhardt, T. Wise, N. Rawashdeh, and S. Paheding, "Gesture controlled collaborative robot arm and lab kit," in *2022 CIEC*, 2024. [Online]. Available: https://peer.asee.org/45693.pdf

[3] M.-V. Drăgoi, A.-V. Frimu, A. Postelnicu, R.-A. Puiu, G. Petrea, and A. Hank, "Interactive teleoperation of an articulated robotic arm using vision-based human hand tracking," *Biomimetics*, vol. 11, no. 2, 2026. [Online]. Available: https://doi.org/10.3390/biomimetics11020151

[4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2006.10214

[5] Z. Chen, Y. J. Yoon, R. Bautista-Montesano, Z. Zhao, A. Mandlekar, and J. Liu, "Teleoplab: Accessible and intuitive teleoperation of a robotic manipulator for remote labs," 2025. [Online]. Available: https://arxiv.org/abs/2509.05547

[6] S. Dass, W. Ai, Y. Jiang, S. Singh, J. Hu, R. Zhang, P. Stone, B. Abbatematteo, and R. Martín-Martín, "Telemoma: A modular and versatile teleoperation system for mobile manipulation," 2024. [Online]. Available: https://arxiv.org/abs/2403.07869

[7] MoveIt!, "Trac-ik kinematics solver." [Online]. Available: https://docs.ros.org/en/kinetic/api/moveit_tutorials/html/doc/trac_ik/trac_ik_tutorial.html

[8] Z. Wang, I. Reed, and A. M. Fey, "Toward intuitive teleoperation in surgery: Human-centric evaluation of teleoperation algorithms for robotic needle steering," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5799–5806. [Online]. Available: https://doi.org/10.1109/ICRA.2018.8460729

[9] C. Meeker, T. Rasmussen, and M. Ciocarlie, "Intuitive hand teleoperation by novice operators using a continuous teleoperation subspace," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5821–5827. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8460506

[10] C. Meeker, M. Haas-Heger, and M. Ciocarlie, "A continuous teleoperation subspace with empirical and algorithmic mapping algorithms for nonanthropomorphic hands," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 1, pp. 373–386, 2022. [Online]. Available: https://doi.org/10.1109/TASE.2020.3035156

[11] R. Li, H. Wang, and Z. Liu, "Survey on mapping human hand motion to robotic hands for teleoperation," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 5, pp. 2647–2665, 2022. [Online]. Available: https://doi.org/10.1109/TCSVT.2021.3057992

[12] Y. Su, X. Chen, T. Zhou, C. Pretty, and G. Chase, "Mixed reality-integrated 3d/2d vision mapping for intuitive teleoperation of mobile manipulator," *Robotics and Computer-Integrated Manufacturing*, vol. 77, p. 102332, 2022. [Online]. Available: https://doi.org/10.1016/j.rcim.2022.102332

[13] M. A. Simão, O. Gibaru, and P. Neto, "Online recognition of incomplete gesture data to interface collaborative robots," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 12, pp. 9372–9382, 2019. [Online]. Available: https://doi.org/0.1109/TIE.2019.2891449

[14] S. Patong and N. Suresh, "Robotic motion planning using the jacobian matrix-based inverse kinematics algorithm," *Association Journal of Interdisciplinary Technics in Engineering Mechanics*, vol. 3, no. 3, pp. 14–21, 2025. [Online]. Available: https://ajitem.org/index.php/journal/article/view/EM33003

[15] S.-O. Park, M. C. Lee, and J. Kim, "Trajectory planning with collision avoidance for redundant robots using jacobian and artificial potential field-based real-time inverse kinematics," *International Journal of Control, Automation and Systems*, vol. 18, pp. 2095–2107, 2020. [Online]. Available: https://doi.org/10.1007/s12555-019-0076-7

[16] Z. Xu, Y. Li, X. Yang, Z. Zhao, J. Zhao, and H. Liu, "A combined inverse kinematics algorithm using fabrik with optimization," *Journal of Intelligent & Robotic Systems*, vol. 108, no. 62, 2023. [Online]. Available: https://doi.org/10.1007/s10846-023-01895-2

[17] A. Aristidou, Y. Chrysanthou, and J. Lasenby, "Extending fabrik with model constraints," *Computer Animation & Virtual Worlds*, vol. 27, pp. 35–57, 2015. [Online]. Available: https://doi.org/10.1002/cav.1630

[18] M. Kelemen, I. Virgala, T. Lipták, L. Miková, F. Filakovský, and V. Bulej, "A novel approach for a inverse kinematics solution of a redundant manipulator," *Applied Sciences*, vol. 8, no. 11, 2018. [Online]. Available: https://doi.org/10.3390/app8112229

[19] N. Andrei, *Sequential Quadratic Programming*. Cham: Springer International Publishing, 2022, pp. 521–567. [Online]. Available: https://doi.org/10.1007/978-3-031-08720-2_15

[20] Y. Bai and S.-J. Hsieh, "A hybrid method using fabrik and custom ann in solving inverse kinematic for generic serial robot manipulator," *The International Journal of Advanced Manufacturing Technology*, vol. 130, pp. 4883–4904, 2024. [Online]. Available: https://doi.org/10.1007/s00170-023-12928-3

[21] G. Fang, Y. Tian, Z.-X. Yang, J. M. P. Geraedts, and C. C. L. Wang, "Efficient jacobian-based inverse kinematics with sim-to-real transfer of soft robots by learning," *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 6, pp. 5296–5306, 2022. [Online]. Available: https://doi.org/10.1109/TMECH.2022.3178303

[22] T. B. Moeslund and E. Granum, "A survey of computer vision-based human motion capture," *Computer Vision and Image Understanding*, vol. 81, no. 3, pp. 231–268, 2001. [Online]. Available: https://doi.org/10.1006/cviu.2000.0897

[23] A. Bhoi, "Monocular depth estimation: A survey," 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1901.09402

[24] S. Saxena, A. Kar, M. Norouzi, and D. J. Fleet, "Monocular depth estimation using diffusion models," 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.14816

[25] Y. Gounine, P. K. Minaoui, and P. M. Elaroussi, "Cybersecurity in robotic telesurgery: Securing robot-console communications for safe remote surgery," in *2025 International Conference on Intelligent Systems: Theories and Applications (SITA)*, 2025, pp. 1–8. [Online]. Available: https://doi.org/10.1109/SITA67914.2025.11273620

[26] A. Bernadotte, "Cyber security for surgical remote intelligent robotic systems," in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, pp. 65–69. [Online]. Available: https://doi.org/10.1109/ICARA56516.2023.10126050

[27] S. Iqbal, S. Farooq, K. Shahzad, A. W. Malik, M. M. Hamayun, and O. Hasan, "Securesurginet: A framework for ensuring security in telesurgery," *International*

Journal of Distributed Sensor Networks, vol. 15, no. 9, p. 1550147719873811, 2019. [Online]. Available: https://doi.org/10.1177/1550147719873811

[28] M. S. Mahmoud and M. M. Hamdan, "Control of teleoperation systems in the presence of cyber attacks: A survey," *IAES International Journal of Robotics and Automation (IJRA)*, vol. 10, no. 3, pp. 235–260, 2021. [Online]. Available: https://doi.org/10.11591/ijra.v10i3.pp235-260

[29] M. Sabouri, "Cybersecurity of teleoperated quadruped robots: A systematic survey of vulnerabilities, threats, and open defense gaps," 2026. [Online]. Available: https://arxiv.org/abs/2602.23404

[30] H. Takanashi, A. Kosugi, K. Teranishi, T. Mizuya, K. Abe, and K. Kogiso, "Cyber-secure teleoperation with encrypted four-channel bilateral control," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 6, pp. 2163–2177, 2025. [Online]. Available: https://doi.org/10.1109/TCST.2025.3577536

[31] D.-F. Hriţcan, A. Graur, D.-G. Bălan, and E.-M. Timofte, "A performance analysis of vpn technologies used in an iot environment," *Advances in Electrical and Computer Engineering*, vol. 25, no. 2, 2025. [Online]. Available: https://www.researchgate.net/profile/Edi-Timofte/publication/393410400_A_Performance_Analysis_of_VPN_Technologies_Used_in_an_IoT_Environment/links/686beba292697d42903d9e91/A-Performance-Analysis-of-VPN-Technologies-Used-in-an-IoT-Environment.pdf

[32] H. Jumakhan and A. Mirzaeinia, "Wireguard: An efficient solution for securing iot device connectivity: Regular research paper (csci-rtmc)," in *2023 International Conference on Computational Science and Computational Intelligence (CSCI)*, 2023, pp. 934–940. [Online]. Available: https://doi.org/10.1109/CSCI62032.2023.00156

[33] C. M. Purba and P. D. Ibnugraha, "Securing data transmission in iot devices using wireguard vpn," in *2025 IEEE International Conference on Artificial Intelligence and Mechatronics Systems (AIMS)*, 2025, pp. 1–5. [Online]. Available: https://doi.org/10.1109/AIMS66189.2025.11229500

[34] *ESP32-WROOM-32 Datasheet*, Espressif, September 2026, rev 3.8. [Online]. Available: https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf

[35] Google, "Mediapipe hand landmarks detection." [Online]. Available: https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker

[36] Ultralytics, "Ultralytics on github." [Online]. Available: https://github.com/ultralytics/ultralytics

[37] T. T. Nguyen, C. Wilson, I. Khan, and J. Dalins, "Object detection approaches to identifying hand images with high forensic values," *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2024. [Online]. Available: https://doi.org/10.1109/SMC54092.2024.10831640

[38] B. Kristianto, C. Dewi, H. D. Purnomo, K. D. Hartomo, and S. Z. M. Hashim, "Utilizing the yolov8 model for accurate hand recognition with complex background," *PeerJ Computer Science*, 2025. [Online]. Available: https://doi.org/10.7717/peerj-cs.3244

[39] J. Docekal, J. Rozlivek, J. Matas, and M. Hoffmann, "Human keypoint detection for close proximity human-robot interaction," in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, 2022, pp. 450–457. [Online]. Available: https://doi.org/10.1109/Humanoids53995.2022.10000133

[40] L. A. O. Moreno and J. H. A. Alcántara, "An adaptation of fabrik algorithm for serial robot's inverse kinematics," in *2022 IEEE International Conference on Engineering Veracruz (ICEV)*, 2022, pp. 1–6. [Online]. Available: https://doi.org/10.1109/ICEV56253.2022.9959663

[41] A. Aristidou and J. Lasenby, "Fabrik: A fast, iterative solver for the inverse kinematics problem," *Graphical Models*, vol. 73, no. 5, pp. 243–260, 2011. [Online]. Available: https://doi.org/10.1016/j.gmod.2011.05.003

Appendix A

Additional Files and Schematics

Add any information here that you would like to have in your project but is not necessary in the main text. Remember to refer to it in the main text. Separate your appendices based on what they are for example. Equation derivations in Appendix A and code in Appendix B etc.

IMPORTANT: Appendix A (see the table below) should provide a summary of how you have met the GAs associated with the course, and where in the report the evidence can be found.

If you have used AI in writing your report, you need to provide details in one of the appendices of how you used it.

If appropriate, in a subsequent appendix, provide a link to any GitHub repository where the code and additional materials for your project can be found.

GA	Requirement	Justification and section in the report
1	Problem-solving	Description here
4	Investigations, experiments and data analysis	Description here
5	Use of engineering tools	Description here
6	Professional and technical communication (Long report)	Description here
8	Individual work	Description here
9	Independent learning ability	Description here

Appendix B

Addenda

B.1 Ethics Forms

Appendix C

Inverse Kinematics Pseudocode

Algorithm 2 Move Joint under Forward Prismatic Constraints

Require: An array J of joints, and the index j of the joint being moved,
Ensure: Joint $J[j]$ is positioned & oriented under the forward prismatic constraints of $J[j+1]$

function FORWARDPRISMATIC(J, j)
    // Shorthand for "previous joint"
    $P \leftarrow J[j+1]$

    // Update joint orientation
    $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
    $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}$

    // Calculate joint length
    $\vec{d} \leftarrow P.\vec{pos} - J[j].\vec{pos}$
    $P.value \leftarrow (\vec{d} \cdot J[j].\vec{axis}_{link}) - J[j].len_{link}$

    // Check joint limits
    if P is limited **then**
        if $P.value > P.max$ **then**
            $P.value \leftarrow P.max$
        else if $P.value < P.min$ **then**
            $P.value \leftarrow P.min$
        end if
    end if

    // Update joint position
    $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link}(P.value + J[j].len_{link})$
end function

Algorithm 3 Move Joint under Forward Hinge Constraints

Require: An array J of joints, and the index j of the joint being moved
Ensure: Joint $J[j]$ is positioned & oriented under the forward hinge constraints of $J[j+1]$

function ForwardHinge(J, j, dir)
    // Shorthand for "previous joint"
    $P \leftarrow J[j+1]$

    // Project distance $\vec{d}$ onto the plane perpendicular to $P.\vec{axis}_{aux}$
    $\vec{d} \leftarrow P.\vec{pos} - J[j].\vec{pos}$
    $\vec{l} \leftarrow \vec{d} - (\vec{d} \cdot P.\vec{axis}_{aux})P.\vec{axis}_{aux}$

    // Update joint orientation
    $mag_l \leftarrow |\vec{l}|$
    if $mag_l = 0$ **then**
        $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
    else
        $J[j].\vec{axis}_{link} \leftarrow (\vec{l}/mag_l)$
    end if
    $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}$

    // Check joint limits
    $sign \leftarrow \text{sgn}((J[j].\vec{axis}_{link} \times P.\vec{axis}_{link}) \cdot J[j].\vec{axis}_{aux})$
    $P.value \leftarrow \cos^{-1}(J[j].\vec{axis}_{link} \cdot P.\vec{axis}_{link})sign$
    if P is limited **then**
        if $P.value > P.max$ **then**
            $P.value \leftarrow P.max$
            $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}\cos(P.value)$
            $J[j].\vec{axis}_{link} \leftarrow J[j].\vec{axis}_{link} + (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        else if $P.value < P.min$ **then**
            $P.value \leftarrow P.min$
            $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}\cos(P.value)$
            $J[j].\vec{axis}_{link} \leftarrow J[j].\vec{axis}_{link} + (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        end if
    end if

    // Update joint position
    $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link}J[j].len_{link}$
end function

Algorithm 4 Move Joint under Forward Pivot Constraints

Require: An array J of joints, and the index j of the joint being moved
Ensure: Joint $J[j]$ is positioned & oriented under the forward pivot constraints of $J[j+1]$

function ForwardPivot(J, j)
    // Shorthand for "previous joint"
    $P \leftarrow J[j+1]$

    // Update joint position
    $J[j].\vec{axis}_{link} \leftarrow P.\vec{axis}_{link}$
    $J[j].\vec{pos} \leftarrow P.\vec{pos} - J[j].\vec{axis}_{link} J[j].len_{link}$

    // Determine auxiliary axis direction
    $\vec{d} \leftarrow J[j].\vec{pos} - J[j+dir].\vec{pos}$
    $\vec{a} \leftarrow J[j].\vec{axis}_{link} \times \vec{d}$
    $mag_a \leftarrow |\vec{a}|$
    if $mag_a = 0$ **then**
        $\vec{a} \leftarrow P.\vec{axis}_{aux}$
    else
        $\vec{a} \leftarrow \vec{a}/mag_a$
    end if

    // Correct the direction of the auxiliary axis
    $J[j].\vec{axis}_{aux} \leftarrow \vec{a}\,\mathrm{sgn}(J[j].\vec{axis}_{aux} \cdot \vec{a})$

    // Check joint limits
    $sign \leftarrow \mathrm{sgn}((J[j].\vec{axis}_{aux} \times P.\vec{axis}_{aux}) \cdot J[j].\vec{axis}_{link})$
    $P.value \leftarrow \cos^{-1}(J[j].\vec{axis}_{aux} \cdot P.\vec{axis}_{aux})sign$
    if P is limited **then**
        if $P.value > P.max$ **then**
            $P.value \leftarrow P.max$
            $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}\cos(P.value)$
            $J[j].\vec{axis}_{aux} \leftarrow J[j].\vec{axis}_{aux} - (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        else if $P.value < P.min$ **then**
            $P.value \leftarrow P.min$
            $J[j].\vec{axis}_{aux} \leftarrow P.\vec{axis}_{aux}\cos(P.value)$
            $J[j].\vec{axis}_{aux} \leftarrow J[j].\vec{axis}_{aux} - (P.\vec{axis}_{link} \times P.\vec{axis}_{aux})\sin(P.value)$
        end if
    end if
end function